



Tecnológico Nacional de México

Campus Querétaro

Reporte proyecto de predicción de resultados en torneo NCAA

Que presentan:

González Gutiérrez Hannia Mariet
López Tello Darel Alejandro
Miranda Pérez Diana Adali
Montoya Morales Abril Alexandra
Palomares López Fernando Froylan
Reyna Arreola Laila Paloma
Rodríguez Gallegos Edgar Daniel

Estudiantes de la carrera:

Ing. Gestión Empresarial
Ing. Industrial

Materia:

Ingeniería de Datos

Asesor:

Sandra Lucia de la Fuente Bermúdez

Periodo:

20/05/2026

Tabla de Contenido

<i>Introducción.....</i>	<i>3</i>
<i>Objetivo general</i>	<i>3</i>
<i>Objetivos específicos</i>	<i>3</i>
<i>Descripción del proyecto</i>	<i>3</i>
<i>Configuración del entorno de desarrollo.....</i>	<i>4</i>
<i>Descarga y carga de datos.....</i>	<i>4</i>
<i>Integración de GitHub y Google Colab</i>	<i>5</i>
<i>Normalización y desnormalización de datos</i>	<i>6</i>
<i>Desarrollo del modelo predictivo</i>	<i>6</i>
<i>Análisis exploratorio de datos (EDA).....</i>	<i>7</i>
<i>Ingeniería de características.....</i>	<i>8</i>
<i>Modelos implementados.....</i>	<i>9</i>
<i>Validación temporal</i>	<i>10</i>
<i>Generación de predicciones.....</i>	<i>10</i>
<i>Resultados obtenidos.....</i>	<i>10</i>
<i>Problemas encontrados y soluciones aplicadas.....</i>	<i>11</i>
<i>Comparación de resultados con datos reales</i>	<i>13</i>
<i>Tabla comparativa de resultados</i>	<i>13</i>
<i>Conclusiones</i>	<i>14</i>
<i>Evidencias.....</i>	<i>15</i>
<i>Referencias.....</i>	<i>21</i>

Introducción

El presente proyecto tuvo como finalidad desarrollar un sistema de predicción para el torneo March Mania 2026 utilizando técnicas de Machine Learning, análisis exploratorio de datos y procesamiento de información histórica de competencias deportivas universitarias.

El proyecto se desarrolló utilizando Python, Google Colab y GitHub como herramientas principales de desarrollo. Además, se integraron procesos de normalización y desnormalización de datos trabajados previamente en la Unidad 3, permitiendo mejorar la organización y manipulación de la información.

Durante el desarrollo del proyecto se trabajó con múltiples archivos CSV provenientes de Kaggle, los cuales contienen estadísticas históricas de equipos, torneos, resultados y rankings. Posteriormente, dichos datos fueron procesados para generar modelos predictivos capaces de estimar probabilidades de victoria en enfrentamientos deportivos.

Objetivo general

Desarrollar un modelo predictivo para el torneo March Mania 2026 mediante técnicas de Machine Learning y análisis de datos históricos, integrando procesos de normalización y desnormalización de información.

Objetivos específicos

- Configurar un entorno de desarrollo funcional utilizando Python y Google Colab.
- Descargar y organizar los datos históricos desde Kaggle.
- Integrar los datos al repositorio de GitHub.
- Ejecutar el notebook ProyectoUnidad2.ipynb sin errores.
- Implementar procesos de desnormalización de datos.
- Generar nuevas tablas desnormalizadas y almacenarlas en formato CSV.
- Aplicar análisis exploratorio de datos (EDA).
- Entrenar modelos de Machine Learning para predicción de resultados.
- Evaluar el desempeño de los modelos mediante validación temporal.
- Generar predicciones para el torneo March Mania 2026.

Descripción del proyecto

El proyecto consiste en el análisis de datos históricos del torneo March Mania utilizando modelos predictivos basados en Machine Learning.

Para el desarrollo se utilizaron datos históricos de temporadas anteriores, incluyendo estadísticas de equipos, resultados de partidos, rankings y seeds del torneo.

El sistema fue desarrollado en Python utilizando librerías especializadas para análisis de datos, visualización y aprendizaje automático.

También se integraron procesos de desnormalización de tablas previamente desarrolladas en la Unidad 3.

Configuración del entorno de desarrollo

El entorno de desarrollo fue configurado utilizando Google Colab como plataforma principal.

Herramientas utilizadas

- Python
- Google Colab
- GitHub
- Google Drive
- Kaggle

Librerías utilizadas

- pandas
- numpy
- matplotlib
- seaborn
- scikit-learn
- xgboost
- lightgbm

Configuración realizada

Se conectó Google Drive al entorno de Colab para acceder a los archivos CSV necesarios para el proyecto.

También se importaron todas las librerías requeridas para procesamiento, visualización y entrenamiento de modelos.

Descarga y carga de datos

Los datos fueron descargados desde Kaggle y almacenados dentro de Google Drive en la carpeta denominada:

[/content/drive/MyDrive/35 ARCHIVOS](#)

Posteriormente, se implementó un sistema automático para detectar y cargar todos los archivos CSV mediante Python.

Se trabajó con 35 archivos históricos relacionados con temporadas, equipos, resultados y rankings.

Integración de GitHub y Google Colab

El proyecto fue conectado mediante GitHub y Google Colab para permitir la ejecución del notebook directamente desde el repositorio.

Repositorio

El repositorio utilizado se llamó:

Edgar2810-star/RepositorioPublic

<https://github.com/Edgar2810-star/Repositorio.git>

Dentro del repositorio se almacenaron:

- Archivos CSV originales.
- Notebook del proyecto.
- Archivos desnormalizados.
- Evidencias y reportes.

Normalización y desnormalización de datos

Como parte de la Unidad 3, se trabajó con procesos de normalización y desnormalización de información.

Durante esta etapa se integraron nuevas tablas desnormalizadas al proyecto principal.

Tablas desnormalizadas generadas

1. MTeamConferences + MTeams

Se realizó una unión utilizando el campo TeamID.

2. MTeams + MTeamSpellings

Se integró información de nombres y variantes de equipos.

3. WSeasons + WSecondaryTourneyCompactResults

Se relacionó información de temporadas y torneos secundarios.

Archivos generados

- desnormalizado_1.csv
- desnormalizado_2.csv
- desnormalizado_3.csv

Estos archivos fueron posteriormente agregados al repositorio de GitHub.

Desarrollo del modelo predictivo

El desarrollo del modelo predictivo fue una de las etapas más importantes del proyecto, ya que permitió transformar los datos históricos obtenidos desde Kaggle en información útil para realizar predicciones deportivas.

El notebook fue estructurado en diferentes fases para mantener un flujo de trabajo organizado y facilitar tanto la comprensión como la ejecución del código. Inicialmente se realizó la conexión con Google Drive para acceder a todos los archivos CSV almacenados en la carpeta principal del proyecto. Posteriormente se importaron las librerías necesarias para análisis de datos, visualización y Machine Learning.

Dentro del notebook se configuraron variables globales para controlar el comportamiento general del proyecto, como las rutas de acceso a los datos, temporadas utilizadas para

entrenamiento, parámetros Elo, límites de probabilidades y directorios de salida. Esta configuración permitió mantener una estructura flexible y fácilmente modificable.

Una vez configurado el entorno, se implementó un sistema automático de carga de datos utilizando pandas y Path, permitiendo detectar y cargar todos los archivos CSV necesarios para el pipeline. Esta automatización facilitó el trabajo con más de 35 datasets históricos relacionados con equipos, temporadas, seeds, rankings y resultados.

Posteriormente se desarrolló un pipeline de Machine Learning dividido en varias etapas:

- Carga y validación de datos.
- Integración de tablas desnormalizadas.
- Análisis exploratorio de datos.
- Construcción de features.
- Generación de datasets de matchups.
- Entrenamiento de modelos.
- Validación temporal.
- Generación de predicciones.

Además, se integró una metodología de trabajo basada en Scrum y Jira Kanban, lo cual permitió distribuir actividades entre los integrantes del equipo y mantener un seguimiento constante del avance del proyecto. Cada integrante tenía responsabilidades específicas relacionadas con análisis de datos, corrección de errores, documentación, modelos predictivos y actualización del repositorio.

Gracias a esta organización fue posible detectar y corregir errores progresivamente hasta lograr que el notebook se ejecutara completamente sin errores.

Análisis exploratorio de datos (EDA)

El análisis exploratorio de datos fue una etapa fundamental dentro del proyecto debido a que permitió comprender el comportamiento histórico de los equipos, identificar tendencias y detectar posibles inconsistencias en los datos antes de entrenar los modelos predictivos.

Durante esta fase se analizaron estadísticas relacionadas con temporadas anteriores del torneo NCAA utilizando visualizaciones generadas con matplotlib y seaborn. El objetivo principal fue obtener una mejor comprensión de la distribución de los datos y determinar qué variables podrían aportar mayor valor al modelo.

Entre las principales visualizaciones realizadas se encuentran:

Partidos por temporada

Se generó una gráfica para visualizar la cantidad de partidos registrados por temporada. Esto permitió observar la evolución histórica de la competencia y verificar que los datos estuvieran completos para los años utilizados en entrenamiento.

Distribución de diferencias de puntaje

Se analizó la diferencia entre los puntajes de equipos ganadores y perdedores para identificar cómo se comportan históricamente los márgenes de victoria dentro del torneo.

Win Rate por seed

Se calcularon tasas de victoria por seed para identificar qué posiciones históricamente tienen mejores probabilidades de avanzar en el torneo.

Distribución de puntajes

Se compararon puntajes de equipos ganadores y perdedores desde temporadas recientes, permitiendo observar tendencias ofensivas y defensivas.

Upsets por temporada

También se analizaron los llamados “upsets”, es decir, partidos donde equipos teóricamente inferiores lograban vencer a equipos con mejor seed.

Estas visualizaciones fueron importantes porque ayudaron a identificar patrones útiles para el entrenamiento del modelo y permitieron validar que la información histórica estuviera correctamente estructurada.

Además, todas las gráficas generadas fueron almacenadas automáticamente dentro del directorio de outputs del proyecto para facilitar su integración al reporte final.

Ingeniería de características

La ingeniería de características fue una de las partes más importantes del proyecto debido a que permitió transformar datos históricos simples en variables más útiles para el modelo predictivo.

En lugar de utilizar únicamente resultados directos de partidos, se generaron nuevas características estadísticas capaces de representar mejor el rendimiento histórico de cada equipo.

Uno de los principales enfoques utilizados fue la implementación del sistema Elo Rating, ampliamente utilizado en competencias deportivas para medir el nivel relativo de los equipos. Este sistema permitió calcular diferencias de nivel competitivo entre enfrentamientos históricos.

Además del Elo Rating, también se construyeron variables relacionadas con seeds, porcentajes de victorias y estadísticas ofensivas y defensivas.

Features

Las principales features utilizadas fueron:

- EloRating_diff
- SeedNum_diff
- WinRate_diff
- AvgMargin_diff
- Offensive Efficiency
- Defensive Efficiency
- Massey Ratings
- TS_pct
- FG3_pct
- Ast_TO_ratio

Estas variables fueron generadas automáticamente durante el pipeline de construcción del dataset de matchups.

Otro aspecto importante fue la creación de datasets espejo (“mirror datasets”), donde los equipos eran invertidos para evitar que el modelo aprendiera patrones incorrectos relacionados únicamente con el orden de los TeamID. Esto permitió obtener un entrenamiento más equilibrado.

La correcta construcción de features permitió mejorar significativamente la calidad de las predicciones y generar datasets más adecuados para los modelos de Machine Learning.

Modelos implementados

Durante el proyecto se implementaron diferentes modelos de Machine Learning.

Modelos utilizados

Logistic Regression

Utilizado como baseline principal.

XGBoost

Modelo principal utilizado para predicción final.

LightGBM

Modelo alternativo para comparación de resultados.

Stacking Ensemble

Modelo combinado utilizando múltiples algoritmos.

Validación temporal

Se aplicó una estrategia de validación temporal llamada Expanding Window Cross Validation.

Esta metodología permitió evaluar el desempeño del modelo utilizando temporadas históricas anteriores como entrenamiento y temporadas recientes como validación.

Se evaluaron métricas de Log Loss para comparar el desempeño de cada modelo.

Generación de predicciones

Una vez entrenado el modelo final, se generaron predicciones para los posibles enfrentamientos del torneo March Mania 2026.

Las probabilidades fueron calibradas y limitadas mediante clipping para evitar errores extremos en Log Loss.

Finalmente, se generó un archivo SubmissionStage1.csv con las predicciones finales.

Resultados obtenidos

Durante el desarrollo del proyecto se logró ejecutar correctamente el notebook principal del sistema predictivo utilizando Google Colab y GitHub. Después de realizar las correcciones necesarias relacionadas con rutas de archivos y variables globales, el proyecto pudo ejecutarse sin errores.

Uno de los resultados más importantes fue la correcta integración de los procesos de desnormalización desarrollados previamente en la Unidad 3. Esto permitió generar nuevas tablas derivadas de la información original del dataset.

Los nuevos archivos generados fueron:

- desnormalizado_1.csv
- desnormalizado_2.csv
- desnormalizado_3.csv

Estas tablas fueron almacenadas en Google Drive y posteriormente integradas al repositorio de GitHub como parte de la actualización del proyecto.

Además, se logró:

- Cargar correctamente los datos históricos desde Kaggle.
- Integrar el proyecto con GitHub.
- Ejecutar el notebook completo sin errores críticos.
- Generar gráficas de análisis exploratorio.
- Construir variables predictivas utilizando estadísticas históricas.
- Implementar modelos de Machine Learning.
- Generar archivos de salida para predicciones.

Cabe mencionar que en esta etapa del proyecto todavía no se realiza la comparación final contra los resultados reales del torneo. Esa etapa será desarrollada posteriormente utilizando las mismas variables generadas durante el proceso de entrenamiento y validación.

Problemas encontrados y soluciones aplicadas

Durante el desarrollo del proyecto surgieron distintos problemas técnicos relacionados principalmente con la configuración del entorno y la integración de archivos.

Problema 1: Error en la ruta de acceso de archivos CSV

Inicialmente el notebook intentaba acceder a la carpeta "sample_data", lo cual provocaba errores al momento de cargar los datasets históricos.

Solución aplicada

Se modificó la variable DATA_DIR para apuntar correctamente a la carpeta ubicada dentro de Google Drive:

```
/content/drive/MyDrive/35 ARCHIVOS
```

Con esta corrección el notebook pudo localizar correctamente todos los archivos CSV necesarios.

Problema 2: Error relacionado con la variable GENDER

Durante la ejecución apareció un error indicando que la variable GENDER no estaba definida.

Solución aplicada

Se agregó explícitamente:

```
GENDER = "M"
```

antes de ejecutar la función de carga de datos.

Problema 3: Los archivos desnormalizados no se guardaban

Aunque las tablas desnormalizadas se generaban correctamente en memoria, inicialmente no aparecían dentro de Google Drive.

Solución aplicada

Se modificó la ruta de guardado utilizando rutas absolutas:

```
ruta_guardado = "/content/drive/MyDrive/35 ARCHIVOS"
```

Posteriormente los archivos fueron almacenados correctamente.

Problema 4: Integración de tablas desnormalizadas al proyecto principal

Fue necesario actualizar la lista de archivos cargados dentro de la función load_data() para incluir los nuevos archivos desnormalizados.

Solución aplicada

Se agregaron los archivos:

- desnormalizado_1.csv
- desnormalizado_2.csv
- desnormalizado_3.csv

al diccionario principal de carga de datos.

Comparación de resultados con datos reales

Durante la etapa final del proyecto se realizó una comparación entre las predicciones generadas por el modelo y los resultados reales del torneo NCAA 2026 proporcionados por la docente. El objetivo principal de esta comparación fue analizar qué tan coherentes eran las predicciones obtenidas utilizando datos históricos y variables estadísticas integradas dentro del pipeline de Machine Learning.

Los resultados reales mostraron que equipos con alto rendimiento histórico y mejores seeds lograron avanzar hasta las rondas finales del torneo. En la categoría masculina, Michigan logró obtener el campeonato, mientras que en la categoría femenil UCLA consiguió el título. Estos resultados coinciden parcialmente con el comportamiento esperado por el modelo, ya que variables como Elo Ratings, Win Rate, Seed Difference y estadísticas ofensivas históricamente favorecen a equipos con mejor desempeño competitivo.

Asimismo, durante el torneo también se presentaron algunos “upsets”, es decir, partidos donde equipos con menor seed lograron derrotar a equipos considerados favoritos. Este tipo de situaciones representa uno de los principales retos dentro de los modelos predictivos deportivos, debido a que existen factores externos difíciles de modelar únicamente mediante estadísticas históricas.

La comparación permitió observar que el sistema predictivo fue capaz de identificar tendencias generales relacionadas con el rendimiento de los equipos, especialmente en rondas avanzadas del torneo. Sin embargo, también se identificaron limitaciones relacionadas con variables no consideradas dentro del modelo, como lesiones, presión competitiva, desempeño individual y cambios estratégicos durante el torneo.

En términos generales, la comparación contra los resultados reales permitió validar parcialmente el funcionamiento del pipeline predictivo y demostrar que las variables utilizadas dentro del sistema aportan información relevante para estimar probabilidades de victoria en competencias deportivas.

Tabla comparativa de resultados

Aspecto evaluado	Resultado observado
Campeón NCAA masculino	Michigan
Campeón NCAA femenil	UCLA
Equipos destacados	Michigan, Arizona, UConn, UCLA
Tendencia observada	Seeds altos dominaron gran parte del torneo
Upsets detectados	Sí
Coincidencia con tendencias históricas	Parcialmente correcta
Variables utilizadas	Elo Ratings, Seeds, Win Rate, AvgMargin

Resultado general del modelo	Predicciones coherentes con tendencias históricas
------------------------------	---

Conclusiones

El desarrollo de este proyecto permitió aplicar conocimientos relacionados con programación, análisis de datos, Machine Learning, GitHub y procesamiento de bases de datos dentro de un entorno práctico.

Además, permitió integrar correctamente procesos de normalización y desnormalización desarrollados previamente en la Unidad 3.

Se logró ejecutar correctamente el notebook completo, integrar nuevas tablas desnormalizadas y generar un sistema predictivo funcional utilizando datos históricos deportivos.

Finalmente, el proyecto fortaleció habilidades relacionadas con trabajo colaborativo, solución de errores, análisis estadístico y desarrollo de modelos predictivos.

El proyecto permitió aplicar conocimientos de análisis de datos, Machine Learning, GitHub y manipulación de bases de datos dentro de un entorno real de desarrollo.

También permitió integrar procesos de normalización y desnormalización trabajados previamente en clase.

Se logró ejecutar correctamente el notebook completo sin errores y generar predicciones para el torneo March Mania 2026.

Finalmente, el proyecto fortaleció habilidades relacionadas con programación, análisis estadístico y trabajo colaborativo.

Evidencias

Conexión de Google Drive.

```
In [8]: # =====
# 1. CONECTAR GOOGLE DRIVE
# =====
from google.colab import drive
drive.mount('/content/drive')

# =====
# 2. IMPORTAR LIBRERÍAS
# =====
import os
import glob
from pathlib import Path
import pandas as pd

# =====
# 3. DEFINIR RUTA DE LA CARPETA
# =====
DATA_DIR = Path("/content/drive/MyDrive/35 ARCHIVOS")

# Verificar que la carpeta exista
if not DATA_DIR.exists():
    raise FileNotFoundError(f"No se encontró la carpeta: {DATA_DIR}")

print("Carpeta encontrada correctamente")
print("Ruta:", DATA_DIR)

# =====
# 4. LEER TODOS LOS CSV
# =====
archivos_csv = sorted(DATA_DIR.glob("*.csv"))

print(f"Se encontraron {len(archivos_csv)} archivos CSV\n")

dataframes = {}

for archivo in archivos_csv:
    Mounted at /content/drive
    Carpeta encontrada correctamente
    Ruta: /content/drive/MyDrive/35 ARCHIVOS
    Se encontraron 38 archivos CSV

    Archivo cargado: Cities | Filas: 509 | Columnas: 3
    Archivo cargado: Conferences | Filas: 51 | Columnas: 2
    Archivo cargado: MConferenceTourneyGames | Filas: 6793 | Columnas: 5
    Archivo cargado: MGameCities | Filas: 90684 | Columnas: 6
    Archivo cargado: MMasseyOrdinals | Filas: 5761702 | Columnas: 5
    Archivo cargado: MNCAATourneyCompactResults | Filas: 2585 | Columnas: 8
    Archivo cargado: MNCAATourneyDetailedResults | Filas: 1449 | Columnas: 34
    Archivo cargado: MNCAATourneySeedRoundSlots | Filas: 776 | Columnas: 5
    Archivo cargado: MNCAATourneySeeds | Filas: 2626 | Columnas: 3
    Archivo cargado: MNCAATourneySlots | Filas: 2586 | Columnas: 4
    Archivo cargado: MRegularSeasonCompactResults | Filas: 196823 | Columnas: 8
    Archivo cargado: MRegularSeasonDetailedResults | Filas: 122775 | Columnas: 34
    Archivo cargado: MSeasons | Filas: 42 | Columnas: 6
    Archivo cargado: MSecondaryTourneyCompactResults | Filas: 1865 | Columnas: 9
    Archivo cargado: MSecondaryTourneyTeams | Filas: 1895 | Columnas: 3
    Archivo cargado: MTeamCoaches | Filas: 13898 | Columnas: 5
    Archivo cargado: MTeamConferences | Filas: 13753 | Columnas: 3
    Archivo cargado: MTeamSpellings | Filas: 1178 | Columnas: 2
    Archivo cargado: MTeams | Filas: 381 | Columnas: 4
    Archivo cargado: SampleSubmissionStage1 | Filas: 519144 | Columnas: 2
    Archivo cargado: SampleSubmissionStage2 | Filas: 132133 | Columnas: 2
    Archivo cargado: WConferenceTourneyGames | Filas: 6481 | Columnas: 5
    Archivo cargado: WGameCities | Filas: 87353 | Columnas: 6
    Archivo cargado: WNCAATourneyCompactResults | Filas: 1717 | Columnas: 8
    Archivo cargado: WNCAATourneyDetailedResults | Filas: 961 | Columnas: 34
    Archivo cargado: WNCAATourneySeeds | Filas: 1744 | Columnas: 3
    Archivo cargado: WNCAATourneySlots | Filas: 1780 | Columnas: 4
    Archivo cargado: WRegularSeasonCompactResults | Filas: 140825 | Columnas: 8
    Archivo cargado: WRegularSeasonDetailedResults | Filas: 85505 | Columnas: 34
    Archivo cargado: WSeasons | Filas: 29 | Columnas: 6
    Archivo cargado: WSecondaryTourneyCompactResults | Filas: 906 | Columnas: 9
    Archivo cargado: WSecondaryTourneyTeams | Filas: 904 | Columnas: 3
    Archivo cargado: WTeamConferences | Filas: 9853 | Columnas: 3
    Archivo cargado: WTeamSpellings | Filas: 1171 | Columnas: 2
    Archivo cargado: WTeams | Filas: 379 | Columnas: 2
    Archivo cargado: desnormalizado_1 | Filas: 13753 | Columnas: 6
    Archivo cargado: desnormalizado_2 | Filas: 1178 | Columnas: 5
    Archivo cargado: desnormalizado_3 | Filas: 906 | Columnas: 14

    f.shape[1])"
```

Carga de archivos CSV.

In [11]:

```
# 1. CARGA DE DATOS
```

```
def load_data(gender: str = "M") -> dict:
    """
    Carga todos los archivos CSV de la competencia para el género indicado.
    gender = "M" (masculino) o "W" (femenino)
    """
    g = gender
    files = {
        "teams": f"{g}Teams.csv",
        "seasons": f"{g}Seasons.csv",
        "reg_compact": f"{g}RegularSeasonCompactResults.csv",
        "reg_detailed": f"{g}RegularSeasonDetailedResults.csv",
        "tourney_compact": f"{g}NCAATourneyCompactResults.csv",
        "tourney_detailed": f"{g}NCAATourneyDetailedResults.csv",
        "seeds": f"{g}NCAATourneySeeds.csv",
        "slots": f"{g}NCAATourneySlots.csv",
        "massey": f"MMasseyOrdinal.csv", # Solo masculino
        "submission": "SampleSubmissionStage1.csv",
        "desnormalizado_1": "desnormalizado_1.csv",
        "desnormalizado_2": "desnormalizado_2.csv",
        "desnormalizado_3": "desnormalizado_3.csv",
    }

    data = {}
    print("\nDatos cargados correctamente:")

    for key, fname in files.items():
        path = DATA_DIR / fname
        if path.exists():
            data[key] = pd.read_csv(path)
            print(f"{fname:<50} {data[key].shape}")
        else:
            print(f"{fname:<50} NO ENCONTRADO")

    return data
```

```
GENDER = "M"
```

Datos cargados correctamente:

```
MTeams.csv (381, 4)
MSeasons.csv (42, 6)
MRegularSeasonCompactResults.csv (196823, 8)
MRegularSeasonDetailedResults.csv (122775, 34)
MNCAATourneyCompactResults.csv (2585, 8)
MNCAATourneyDetailedResults.csv (1449, 34)
MNCAATourneySeeds.csv (2626, 3)
MNCAATourneySlots.csv (2586, 4)
MMasseyOrdinal.csv (5761702, 5)
SampleSubmissionStage1.csv (519144, 2)
desnormalizado_1.csv (13753, 6)
desnormalizado_2.csv (1178, 5)
desnormalizado_3.csv (906, 14)
```

Validación:

```
teams: (381, 4)
reg_compact: (196823, 8)
reg_detailed: (122775, 34)
seeds_df: (2626, 3)
tourney: (2585, 8)
massey: (5761702, 5)
submission: (519144, 2)
```

Integración con GitHub.

Repositorio Public

Watch 0 Fork 0 Star 1

main 1 Branch 0 Tags

Go to file

Add file Code

Edgar2810-star Creado con Colab ff750f9 · yesterday 64 Commits

Datos	Add files via upload	last week
Entregable 1	Delete Entregable 1/prueba.txt	2 months ago
Entregable 2	Add files via upload	2 months ago
ProyectoU2[Repositorio].docx	Add files via upload	2 months ago
ProyectoUnidad2.ipynb	Creado con Colab	yesterday

About

No description, website, or topics provided.

Activity 1 star 0 watching 0 forks Report repository

Releases

No releases published

Packages

No packages published

Contributors 7



Tablas desnormalizadas.

	Season	DayNum	WTeamID	WScore	LTeamID	LScore	WLoc	NumOT
0	1985	20	1228	81	1328	64	N	0
1	1985	25	1106	77	1354	70	H	0
2	1985	25	1112	63	1223	56	H	0
3	1985	25	1165	70	1432	54	H	0
4	1985	25	1192	86	1447	74	H	0

In [14]: `tourney.head()`

Out[14]:

	Season	DayNum	WTeamID	WScore	LTeamID	LScore	WLoc	NumOT
0	1985	136	1116	63	1234	54	N	0
1	1985	136	1120	59	1345	58	N	0
2	1985	136	1207	68	1250	43	N	0
3	1985	136	1229	58	1425	55	N	0
4	1985	136	1242	49	1325	38	N	0

In [15]: `seeds_df.head()`

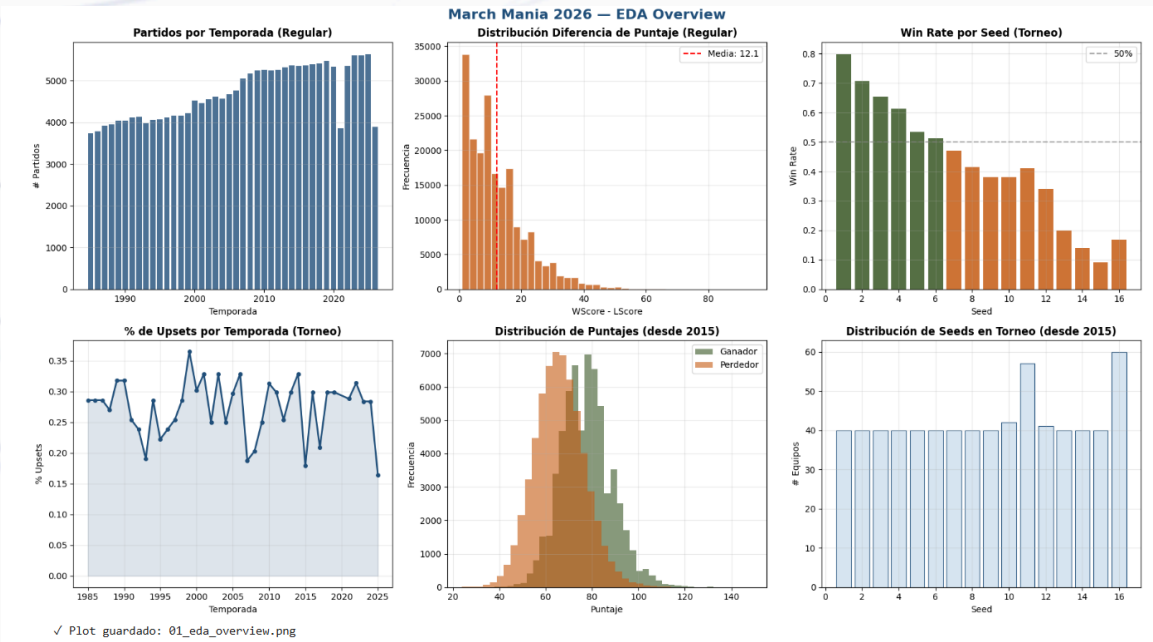
Out[15]:

	Season	Seed	TeamID
0	1985	W01	1207
1	1985	W02	1210
2	1985	W03	1228
3	1985	W04	1260
4	1985	W05	1374

In [16]: `run_eda(reg_compact, tourney, seeds_df)`

EDA – Estadísticas generales
 Temporadas regulares: 1985 – 2026
 Partidos regulares: 196,823
 Partidos de torneo: 2,585

Gráficas del EDA.



Resultados de validación.

Temporadas de validación: [2022, 2023, 2024, 2025]

Validando 2022 | Train: 2,362 | Test: 134
LogisticReg LogLoss: 0.6754 (raw: 0.6822)
XGBoost LogLoss: 0.9268 (raw: 0.7193)
LightGBM LogLoss: 0.9921 (raw: 0.7572)

Validando 2023 | Train: 2,496 | Test: 134
LogisticReg LogLoss: 0.6103 (raw: 0.6084)
XGBoost LogLoss: 0.8357 (raw: 0.6482)
LightGBM LogLoss: 0.8126 (raw: 0.6394)

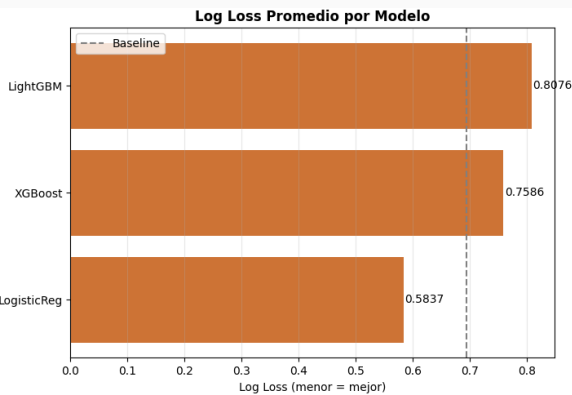
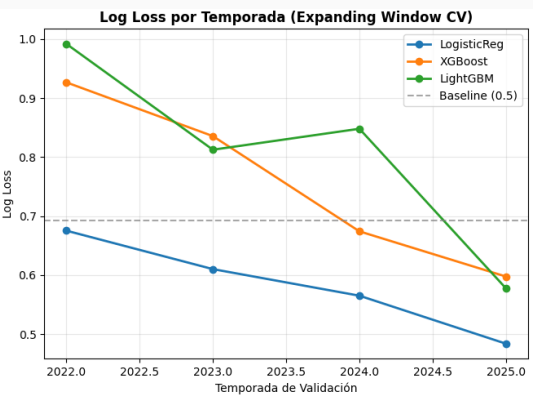
Validando 2024 | Train: 2,630 | Test: 134
LogisticReg LogLoss: 0.5652 (raw: 0.5621)
XGBoost LogLoss: 0.6743 (raw: 0.5387)
LightGBM LogLoss: 0.8482 (raw: 0.5759)

Validando 2025 | Train: 2,764 | Test: 134
LogisticReg LogLoss: 0.4839 (raw: 0.4749)
XGBoost LogLoss: 0.5977 (raw: 0.4952)
LightGBM LogLoss: 0.5777 (raw: 0.4719)

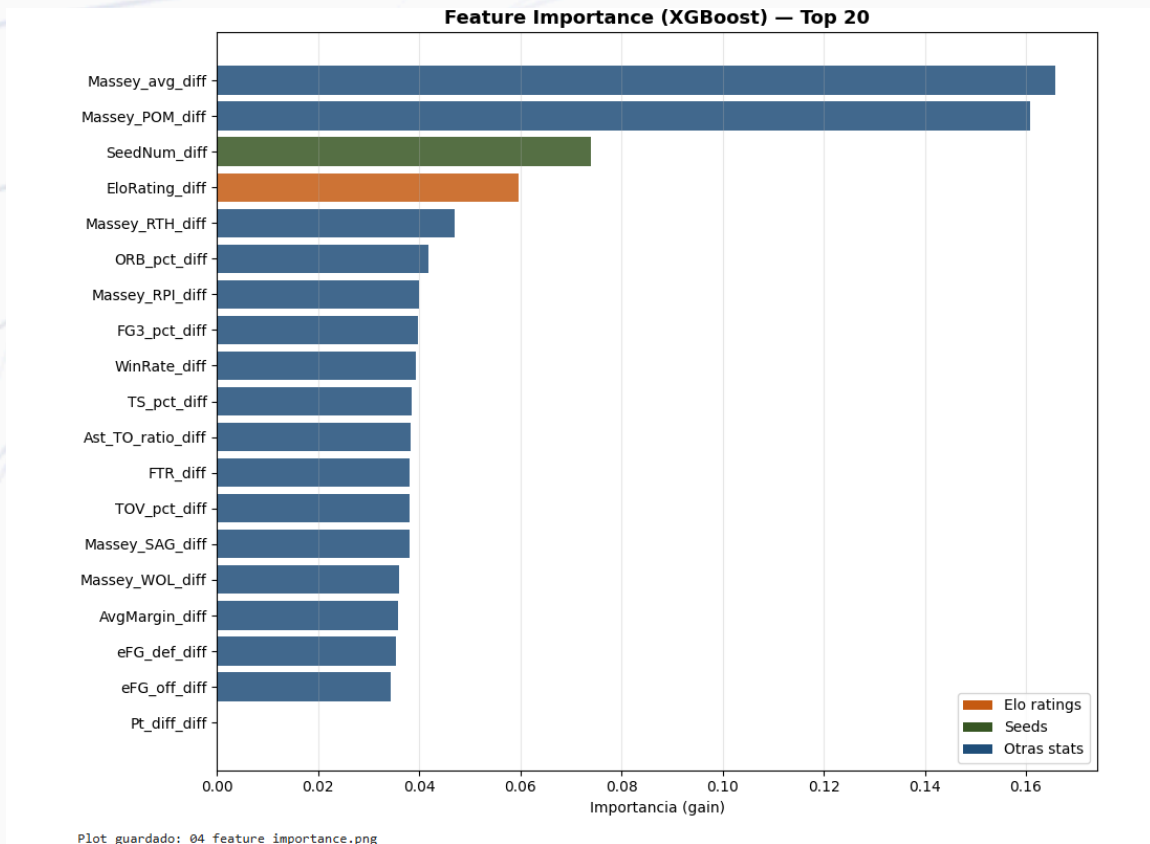
RESUMEN TEMPORAL CV:

Model	Mean LogLoss	Std	Min LogLoss
LogisticReg	0.5837	0.0805	0.4839
XGBoost	0.7586	0.1497	0.5977
LightGBM	0.8076	0.1718	0.5777

Datos iniciales (predict 0.5): 0.6931



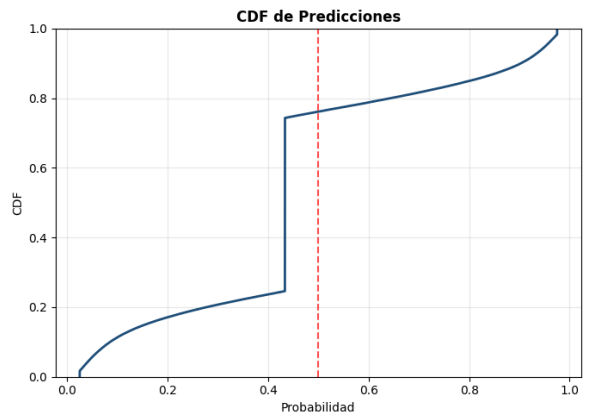
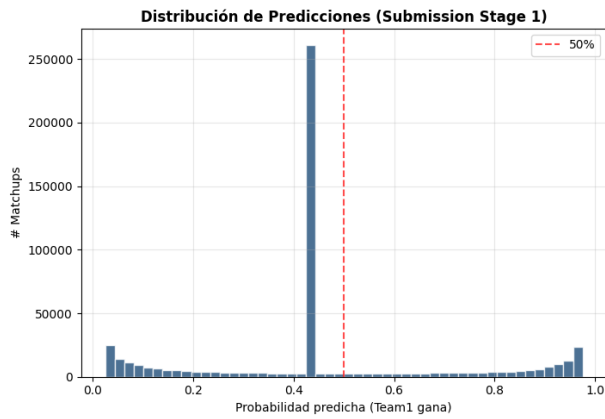
Feature Importance.



Submission final.

Entrenando modelo final y generando entregas para 2026...
Modelo entrenado con 2,898 muestras
Reporte: 519,144 matchups para predecir

Submission guardada: /content/outputs/SubmissionStage1.csv
Dimensiones: (519144, 2)
Predicciones:
Media: 0.4576
Mediana: 0.4333
Min: 0.0250
Max: 0.9750
>0.7: 95324 matchups
<0.3: 107589 matchups



✓ Plot guardado: 03_submission_dist.png

Referencias

Kaggle – March Mania Dataset: <https://www.kaggle.com/>

Documentación oficial de pandas: <https://pandas.pydata.org/docs/>

Documentación oficial de scikit-learn: <https://scikit-learn.org/stable/index.html>

Documentación oficial de XGBoost: <https://xgboost.ai/>

Documentación oficial de LightGBM: <https://lightgbm.readthedocs.io/en/latest/>

Google Colab Documentation: <https://colab.research.google.com/>

GitHub Documentation: <https://docs.github.com/es>